

Web Communication & RESTful APIs

Today

We will be unpacking some terms

API

REST / RESTful

JSON

We will demonstrate an example
application with Flask

Computers Need to Talk to Each Other

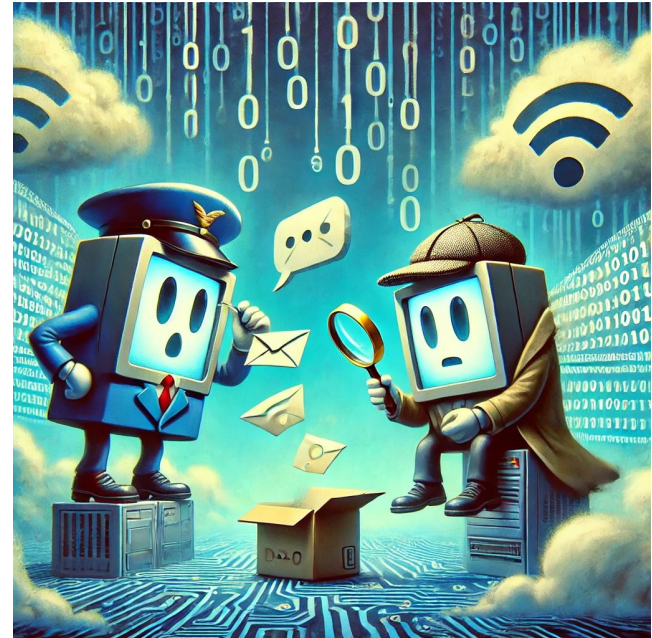
Exchanging information is not trivial.

Even on the same machine different programs need to talk to each other.

An API (Application Programming Interface) is a set of rules and protocols that allow different software applications to communicate.

APIs can be local (within a system)
or remote (across networks).

They can be language-specific (eg Python APIs)
or platform-independent (eg RESTful web APIs)



REST

Representational State Transfer

A loose set of rules with the following core principles:

- Server-client architecture

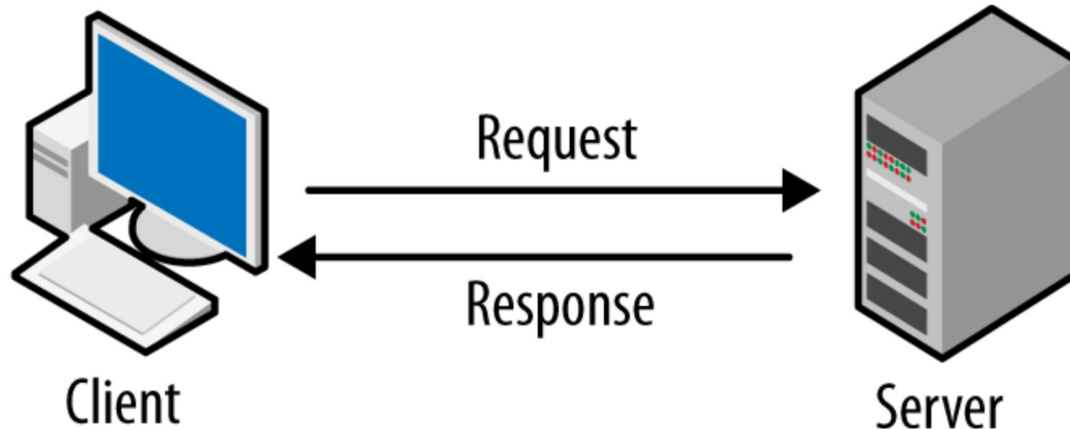
- Stateless

- Uniform Interface

An API that adheres to these rules is called RESTful

Client-Server Architecture

- ✓ Client Requests Data → Sends an HTTP request.
- ✓ Server Processes the Request → Retrieves data or performs logic.
- ✓ Server Sends a Response → Returns data (often in JSON format).
- ✓ Decoupled & Scalable → Clients and servers work independently.



Stateless

Stateless is a weird thing to say about something called Representational State Transfer.

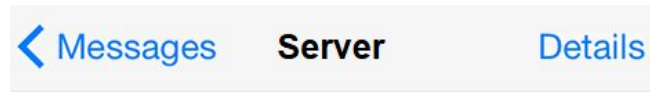
The key is that the communication is stateless.

Every message contains all necessary information.



Give me a large taleggio salsiccia pizza

Here you go 🍕



Give me a pizza

Which toppings?

Taleggio salsiccia

???

Uniform Interface

Requests will always be for resources accessible through an URL.
Client interacts with these resources through a representation (e.g. JSON, XML).

HTTP requests match up with CRUD operations.

GET → Retrieve data

POST → Create new data

PUT → Update existing data

DELETE → Remove data

JSON

JSON (JavaScript Object Notation) is a text-based data format.
You and computers can both read it pretty easily.
Very similar to what you know from Python.

Objects: Represented by curly braces `{}` and contain key-value pairs.

Arrays: Represented by square brackets `[]` and contain ordered lists of values.

JSON

example.py > ...

```
1 import json
2
3 # Load JSON from file
4 with open('example.json', 'r') as f:
5     data = json.load(f)
6
7 print("Loaded from JSON file:")
8 print(data)
9
10 # Equivalent Python dictionary
11 python_dict = {
12     "name": "Alice",
13     "age": 30,
14     "is_student": False,
15     "courses": ["Math", "Science"],
16     "address": {
17         "city": "Amsterdam",
18         "zip": "1012AB"
19     }
20 }
21
22 print("\nPython dictionary:")
23 print(python_dict)
```

example.json > ...

```
1 {
2     "name": "Alice",
3     "age": 30,
4     "is_student": false,
5     "courses": ["Math", "Science"],
6     "address": {
7         "city": "Amsterdam",
8         "zip": "1012AB"
9     }
10 }
```

Flask Examples